

Create and Retrieve with Flask-RESTful (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-flask-restful-cr>  <https://github.com/learn-co-curriculum/python-p4-flask-restful-cr/issues/new>

Learning Goals

- Build RESTful APIs that are easy to navigate and use in applications.
-

Key Vocab

- **Representational State Transfer (REST)**: a convention for developing applications that use HTTP in a consistent, human-readable, machine-readable way.
 - **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
 - **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
 - **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
 - **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
 - **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
 - **PUT** : an HTTP request method that signifies that the client is attempting to update a resource with new information contained in a complete record.
 - **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.
-

Introduction

Flask as you've learned it is already a great tool for building RESTful APIs, but it's important to always seek out the best tools for the job. There are dozens of extensions designed exclusively for use with Flask, and one, [Flask-RESTful](https://flask-restful.readthedocs.io/en/latest/)  [\(https://flask-restful.readthedocs.io/en/latest/\)](https://flask-restful.readthedocs.io/en/latest/), makes it *very* easy to build RESTful APIs.

Flask-RESTful Example

Let's take a look at a bare-bones API built with Flask-RESTful:

```
# example only

from flask import Flask
from flask_restful import Api, Resource

app = Flask(__name__)
api = Api(app)

class Newsletter(Resource):
    def get(self):
        return {"newsletter": "it's a beautiful 108 out in Austin today"}

api.add_resource(Newsletter, '/newsletters')

if __name__ == '__main__':
    app.run(port=5555)
```

We can run this app just like any other- with `python app.py` or `flask run` - then navigate to the client in order to access `Newsletter` 's resources. Since the data here is returned in such a simple format, we can access it most easily from the command line with `curl` :

```
$ curl http://127.0.0.1:5555  
# => {"newsletter: "it's a beautiful 108 out in Austin today"}
```

So what's going on here?

Api and Resource

Flask-RESTful's `Api` class is the constructor for your RESTful API as a whole. It is initialized with a Flask application instance and populates with resources later on. These resources all inherit from the `Resource` class, which includes conditions for throwing exceptions and base methods for each HTTP method that explicitly disallow them.

When `Resource` subclasses are added to the `Api` instance with `add_resource()`, it uses the newly defined HTTP verb instance methods to determine which routes to create at the provided URL.

`Api` and `Resource` aren't the only classes available to us through Flask-RESTful, but they're more than enough to get us started.

What's Missing?

Because the `Resource` class and `create_resource` methods handle tasks normally carried out by the `@app.route()` decorator, we don't need to include the decorator itself. Remember though: if you add any non-RESTful views to your app, you still need `app.route()` !

Getting Started

Enter your virtual environment with `pipenv install; pipenv shell`.

```
pipenv install && pipenv shell
```

If you prefer using a Flask environment to a script, enter the `server/` directory and run the following to configure your Flask environment:

```
$ cd server  
$ export FLASK_APP=app.py
```

```
$ export FLASK_RUN_PORT=5555
```

Recall that the first command is not necessary if our app is contained in a file called `app.py` - it's a good habit to build nonetheless.

Open `server/app.py` and enter the following code to create a RESTful home page:

```
#!/usr/bin/env python3

from flask import Flask, request, make_response
from flask_migrate import Migrate
from flask_restful import Api, Resource

from models import db, Newsletter

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///newsletters.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False

migrate = Migrate(app, db)
db.init_app(app)

api = Api(app)

class Home(Resource):

    def get(self):

        response_dict = {
            "message": "Welcome to the Newsletter RESTful API",
        }

        response = make_response(
```

```
        response_dict,  
        200  
    )  
  
    return response  
  
api.add_resource(Home, '/')  
  
if __name__ == '__main__':  
    app.run(port=5555, debug=True)
```

Run `flask run` (or `python app.py`) from the `server/` directory and you should see the following at <http://127.0.0.1:5555>  (<http://127.0.0.1:5555>):

```
{  
  "message": "Welcome to the Newsletter RESTful API"  
}
```

Congratulations on creating your first RESTful API endpoint!

You'll notice that there are quite a few imports and lines of configuration that relate to databases- we'll be working with one in this lesson, but the models and migrations have already been created for you. When you're ready, run `flask db upgrade` to create the database and `python seed.py` in the `server` directory to seed it with fake data.

```
$ flask db upgrade  
$ python seed.py
```

Retrieving Records with Flask-RESTful

Our homepage is a perfectly good example of a successful **GET** request, but it doesn't truly allow other people's applications to interact with our newsletter database. Let's set up another route, **/newsletters**, that returns all of the records from the **newsletters** table. Open **server/app.py** and enter the following beneath your **Home** view:

```
# server/app.py

class Newsletters(Resource):

    def get(self):

        response_dict_list = [n.to_dict() for n in Newsletter.query.all()]

        response = make_response(
            response_dict_list,
            200,
        )

        return response

api.add_resource(Newsletters, '/newsletters')
```

While the outside structure of views in Flask-RESTful is quite different from vanilla Flask, the internal structure is virtually the same. We can write each of our views- for create, retrieve, update, and delete- just as we did before. The main difference here is that instead of each HTTP verb getting a code block inside of a view function, they each get an instance method inside of a **Resource** class.

Run **flask run** from the **server/** directory and you should see something similar to the following at <http://127.0.0.1:5555/newsletters>  (<http://127.0.0.1:5555/newsletters>):

```
[
  {
    "body": "Create southern girl news. Image interesting mean professor federal agree. Clearly before seat threat c",
    "edited_at": null,
    "id": 1,
    "published_at": "2022-09-21 18:35:17",
    "title": "Establish they."
  },
  {
    "body": "Really attack we ground production game. Late agency example local break start. Tell leader new above j",
    "edited_at": null,
    "id": 2,
    "published_at": "2022-09-21 18:35:17",
    "title": "Plan wonder manage."
  },
  {
    "body": "Will suffer choice impact. Audience happen write feel represent. Woman discover million kitchen. Althou",
    "edited_at": null,
    "id": 3,
    "published_at": "2022-09-21 18:35:17",
    "title": "Meet cut stuff."
  },
  ...
]
```

Creating Records with Flask-RESTful

Let's move onto creating records with `POST` requests. Reopen `server/app.py` and add the following method to the bottom of the `Newsletters` class:

```
# server/app.py
def post(self):
    new_record = Newsletter(
        title=request.form['title'],
        body=request.form['body'],
    )

    db.session.add(new_record)
    db.session.commit()

    response_dict = new_record.to_dict()

    response = make_response(
        response_dict,
        201,
    )

    return response
```

NOTE: We do NOT need to run the `add_resource()` method twice, as the `GET` and `POST` routes are accessible through the same `Resource` and `URL`.

There's nothing you haven't seen before here: we retrieve form data through the request context, use it to create a new Newsletter record, commit that record to the database, then return it to the client with a 201 status code to denote that it was created successfully.

Try it out for yourself: open Postman and navigate to <http://127.0.0.1:5555/newsletters>  [.\(http://127.0.0.1:5555/newsletters\)](http://127.0.0.1:5555/newsletters). Change the request method to `POST`, edit the `Body > form-data` with a title and body, then hit submit. You should see a response similar to the following:

```
{
  "body": "This is the body of the newsletter entitled \"Mr. Title\".",
  "edited_at": null,
```



```
"id": 51,  
"published_at": "2022-09-21 19:16:31",  
"title": "Mr. Title"  
}
```

NOTE: `form-data` does not require the use of quotes for strings, and will throw an error if you use them.

Building Another Resource and Retrieving a Single Record

We won't always want to read *every* newsletter, so we should probably build a route to get a single record back from the database. There are a couple things to consider before we begin:

1. A `GET` route already exists under `newsletters/`.
2. Retrieving a single record means that we need some sort of `identifier`.

This means that we need to build a new `Resource` for this endpoint, and that it should include the `id` in the URL. Let's give it a shot!

```
# server/app.py
```

```
class NewsletterByID(Resource):  
  
    def get(self, id):  
  
        response_dict = Newsletter.query.filter_by(id=id).first().to_dict()  
  
        response = make_response(  
            response_dict,  
            200,  
        )  
  
        return response
```

```
api.add_resource(NewsletterByID, '/newsletters/<int:id>')
```

Only two differences between this and our original **GET** route:

1. We need to include **id** in our method's arguments and the resource URL.
2. We need to chain some commands together to get the record with the provided **id**.

Check out this lesson's finished product: open Postman and navigate to <http://127.0.0.1:5555/newsletters/20> ➞

<http://127.0.0.1:5555/newsletters/20>. Make sure that your request method is **GET**, then click submit. You should see something similar to the following:

```
{
  "body": "College tax head change. Claim exactly because choose. Church edge center across test stock.",
  "edited_at": null,
  "id": 20,
  "published_at": "2022-09-21 18:35:17",
  "title": "Court probably not."
}
```

Conclusion

Flask-RESTful is a very simple tool that allows us to properly and effectively use HTTP request methods in our applications. Like other extensions, it reduces the amount of code you have to write to accomplish common tasks- and if you don't need to accomplish those common tasks, you can just leave it out!

Solution Code

```
#!/usr/bin/env python3

from flask import Flask, request, make_response
from flask_migrate import Migrate
from flask_restful import Api, Resource

from models import db, Newsletter

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///newsletters.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False

migrate = Migrate(app, db)
db.init_app(app)

api = Api(app)

class Home(Resource):

    def get(self):

        response_dict = {
            "message": "Welcome to the Newsletter RESTful API",
        }

        response = make_response(
            response_dict,
            200
        )

        return response
```

```
api.add_resource(Home, '/')

class Newsletters(Resource):

    def get(self):

        response_dict_list = [n.to_dict() for n in Newsletter.query.all()]

        response = make_response(
            response_dict_list,
            200,
        )

        return response

    def post(self):
        new_record = Newsletter(
            title=request.form['title'],
            body=request.form['body'],
        )

        db.session.add(new_record)
        db.session.commit()

        response_dict = new_record.to_dict()

        response = make_response(
            response_dict,
            201,
        )

        return response
```

```
api.add_resource(Newsletters, '/newsletters')

class NewsletterByID(Resource):

    def get(self, id):

        response_dict = Newsletter.query.filter_by(id=id).first().to_dict()

        response = make_response(
            response_dict,
            200,
        )

        return response

api.add_resource(NewsletterByID, '/newsletters/<int:id>')

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Resources

- [What RESTful Actually Means](https://codewords.recurse.com/issues/five/what-restful-actually-means)  (https://codewords.recurse.com/issues/five/what-restful-actually-means)
- [Flask-RESTful](https://flask-restful.readthedocs.io/en/latest/)  (https://flask-restful.readthedocs.io/en/latest/)
- [HTTP request methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)  (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)

This tool needs to be loaded in a new browser window

Load Create and Retrieve with Flask-RESTful (CodeGrade) in a new window

